

STRUKDAT BAB 2

List Tunggal (Singly Linked List)

1. Pengertian

Singly Linked List adalah struktur data linear yang terdiri dari kumpulan node.

Setiap node memiliki:

- Data
- Pointer ke node berikutnya

Ilustrasi:



Berbeda dengan array:

- Array menggunakan memori berurutan.
- Linked List menggunakan memori dinamis.

2. Keunggulan Linked List

Array:

[10][20][30][40]

Jika ingin menyisipkan data di tengah:

[10][20][25][30][40]

harus menggeser elemen.

Linked List:

10 -> 20 -> 30 -> 40

Cukup mengubah pointer.

3. Struktur Node

Node terdiri dari:

Data

Pointer Next

Implementasi:

```
struct Node{  
  
    int data;  
    struct Node* next;  
  
};
```

4. Head

Head adalah pointer yang menunjuk node pertama.

Contoh:

```
struct Node* head = NULL;
```

Ilustrasi:

```
head  
|  
v  
10 -> 20 -> 30 -> NULL
```

5. Membuat Node Baru

Menggunakan:

`malloc()`

Contoh:

```
struct Node* newNode =  
(struct Node*) malloc(sizeof(struct Node));
```

Mengisi data:

```
newNode->data = 10;  
newNode->next = NULL;
```

6. Traversal

Traversal adalah proses mengunjungi seluruh node.

Contoh:

```
void printList(struct Node* head){  
  
    struct Node* temp = head;  
  
    while(temp != NULL){  
  
        printf("%d ", temp->data);  
  
        temp = temp->next;  
  
    }  
  
}
```

Ilustrasi:

10 -> 20 -> 30 -> NULL

Output:

10 20 30

7. Insert First

Menambahkan node di depan.

Awal:

10 -> 20 -> 30

Tambah:

5

Hasil:

5 -> 10 -> 20 -> 30

Implementasi:

```
void insertFirst(struct Node** head,
                 int data){

    struct Node* newNode =
        (struct Node*) malloc(sizeof(struct Node));

    newNode->data = data;

    newNode->next = *head;

    *head = newNode;

}
```

8. Insert Last

Menambahkan node di belakang.

Awal:

10 -> 20 -> 30

Tambah:

40

Hasil:

10 -> 20 -> 30 -> 40

Implementasi:

```
void insertLast(struct Node** head,
                int data){

    struct Node* newNode =
        (struct Node*) malloc(sizeof(struct Node));

    newNode->data = data;
    newNode->next = NULL;

    if(*head == NULL){

        *head = newNode;
        return;

    }

    struct Node* temp = *head;

    while(temp->next != NULL){

        temp = temp->next;

    }

    temp->next = newNode;

}
```

9. Insert After

Contoh:

Awal:

10 -> 20 -> 30

Sisipkan setelah 20:

25

Hasil:

10 -> 20 -> 25 -> 30

Implementasi:

```
newNode->next = current->next;
```

```
current->next = newNode;
```

10. Delete First

Awal:

10 -> 20 -> 30

Hasil:

20 -> 30

Implementasi:

```
void deleteFirst(struct Node** head){
```

```
    if(*head == NULL)
```

```
        return;

    struct Node* temp = *head;

    *head = (*head)->next;

    free(temp);

}
```

11. Delete Last

Awal:

10 -> 20 -> 30

Hasil:

10 -> 20

Implementasi:

```
while(temp->next->next != NULL){

    temp = temp->next;

}
```

12. Delete Data Tertentu

Contoh:

Awal:

10 -> 20 -> 30 -> 40

Hapus:

30

Hasil:

10 -> 20 -> 40

13. Searching

Contoh:

Cari:

20

Implementasi:

```
int pos = 1;
```

```
while(temp != NULL){
```

```
    if(temp->data == x){
```

```
        return pos;
```

```
    }
```

```
    pos++;
```

```
    temp = temp->next;
```

```
}
```

14. Menghitung Jumlah Node

```
int count = 0;

while(temp != NULL){

    count++;

    temp = temp->next;

}
```

15. Mencari Nilai Maksimum

```
int maks = head->data;
```

```
while(temp != NULL){

    if(temp->data > maks){

        maks = temp->data;

    }

    temp = temp->next;

}
```

16. Mencari Nilai Minimum

```
int min = head->data;
```

```
while(temp != NULL){  
  
    if(temp->data < min){  
  
        min = temp->data;  
  
    }  
  
    temp = temp->next;  
  
}
```

17. Kompleksitas Operasi

Operasi	Kompleksitas
Insert First	O(1)
Insert Last	O(n)
Search	O(n)
Delete First	O(1)
Delete Last	O(n)
Traversal	O(n)

18. Kesalahan Umum

Lupa Alokasi Memori

Salah:

```
struct Node* node;
```

```
node->data = 10;
```

Benar:

```
node =
```

```
(struct Node*) malloc(sizeof(struct Node));
```

Null Pointer

Salah:

```
temp->next->next
```

padahal:

```
temp->next == NULL
```

Memory Leak

Salah:

```
head = head->next;
```

tanpa:

```
free(temp);
```

19. Implementasi Lengkap

```
#include <stdio.h>
#include <stdlib.h>

struct Node{

    int data;
    struct Node* next;

};

void insertLast(struct Node** head,
               int data){

    struct Node* newNode =
        (struct Node*) malloc(sizeof(struct Node));

    newNode->data = data;
    newNode->next = NULL;

    if(*head == NULL){

        *head = newNode;
        return;

    }

    struct Node* temp = *head;

    while(temp->next != NULL){

        temp = temp->next;

    }

    temp->next = newNode;

}
```

```
void printList(struct Node* head){  
  
    while(head != NULL){  
  
        printf("%d ", head->data);  
  
        head = head->next;  
  
    }  
  
}  
  
int main(){  
  
    struct Node* head = NULL;  
  
    insertLast(&head,10);  
    insertLast(&head,20);  
    insertLast(&head,30);  
  
    printList(head);  
  
    return 0;  
}
```
